

Machine Learning

Deutsch: Maschinelles Lernen - Wie ein KI Modell aus Daten Muster lernt, um aus neuen Daten Vorhersagen zu treffen (Inferenz). Im wesentlichen gibt es 3 Arten:

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning (Verstärkungslernen)

⇒ Wir werden hier über Supervised Learning sprechen, mit einem besonderen Schwerpunkt auf Artificial Neural Networks (ANNs)

Supervised Learning

(labeled)

Der Algorithmus wird auf einem beschrifteten Datensatz trainiert.

⇒ Jede Eingabe hat eine bekannte Ausgabe (Beschriftung).

Beispiel: Ein Datensatz zum Trainieren eines Modells zur Erkennung von Spam-E-mails würde mehrere inputs für die Betreffzeile enthalten, wobei die korrekten outputs als Spam oder Nicht-Spam gekennzeichnet wird.

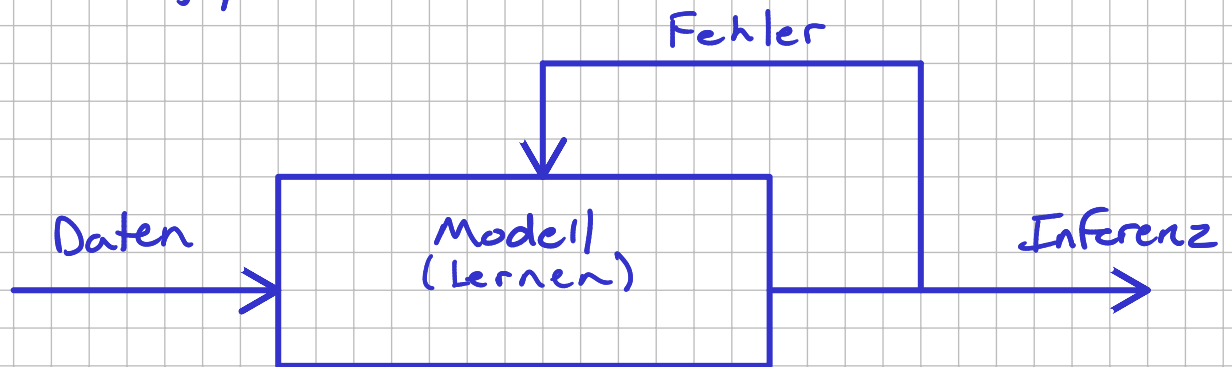
Betreff

Label

"Ihr Konto wurde eingeschränkt!"	Spam
"Sie haben im Lotto gewonnen!"	Spam
"Teamtreffen am 15 Uhr in Raum 201"	Nicht-Spam

Modell - Lern - Inferenz

In unserem Modell - Lern - Inferenz - Paradigma wird überwachtes Lernen durchgeführt, indem Daten eingaben in das Modell eingespeist werden, seine Vorhersagen mit bekannten korrekten Ausgaben verglichen werden, und das Modell dann auf der Grundlage des gemessenen Fehlers angepasst wird.



Arten von Problemen

Zwei Arten von Problemen werden hier behandelt:

- 1) Classification (Klassifizieren)
- 2) Regression

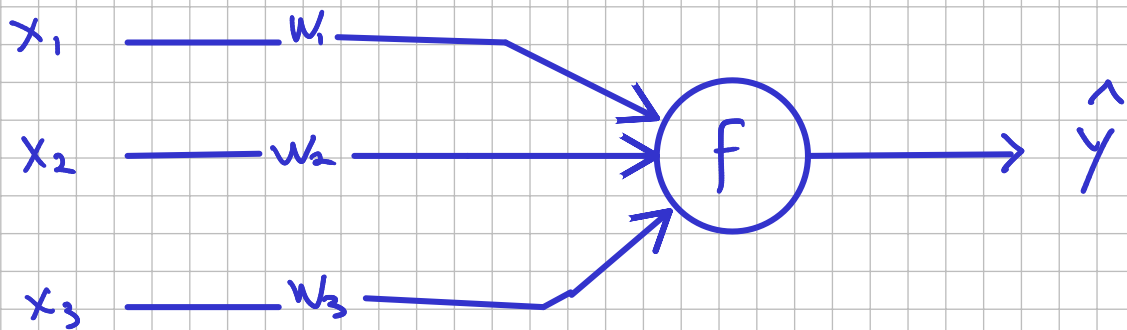
Der Unterschied liegt in der Art der Ausgabe, die das Modell vorhersagt:

	<u>Classification</u>	<u>Regression</u>
Output type	Klasse/Kategorie	Reelle Zahl
Beispiel	Spam (0 oder 1)	12.6 Grad

Perceptron

Bei ANNs wird das Supervised Learning erreicht, indem Daten in die Eingänge des Netzes eingespeist werden, und daraus eine Ausgabe abgeleitet wird. Der Fehler in der Vorhersage wird berechnet und als Parameter für die Anpassung der Gewichte des Modells verwendet.

Das einfachste ANN ist das einzelne Perceptron.



- f ist ein Aktivierungsfunktion
- w ist ein Gewichtsvektor
- x ist ein Eingangsvektor
- $\hat{y}$ ist die Vorhersage (Inferenz)
- y ist ein Labelvektor

* f kann verschiedenen Funktionen sein
z.B. ReLU, Step, Softmax, usw.

Bei einem Perceptron ist die Ausgabe:

$$\hat{y} = f(w^T x + b) \quad * \text{ } b \text{ ist der Bias}$$

Perceptron Classification (Klassifizieren)

Perceptrons sind ideal für Klassifizierungsprobleme, bei denen die Klassen linear trennbar sind.

⇒ Ein binäres Klassifizierungsproblem mit Eingangsvektor x und einem Label y ist linear trennbar, wenn es w und b gibt, dass:

$$\hat{y} = 1 \Rightarrow w^T x + b > 0$$

$$\hat{y} = 0 \Rightarrow w^T x + b < 0$$

Beispiel: Autonomes Auto

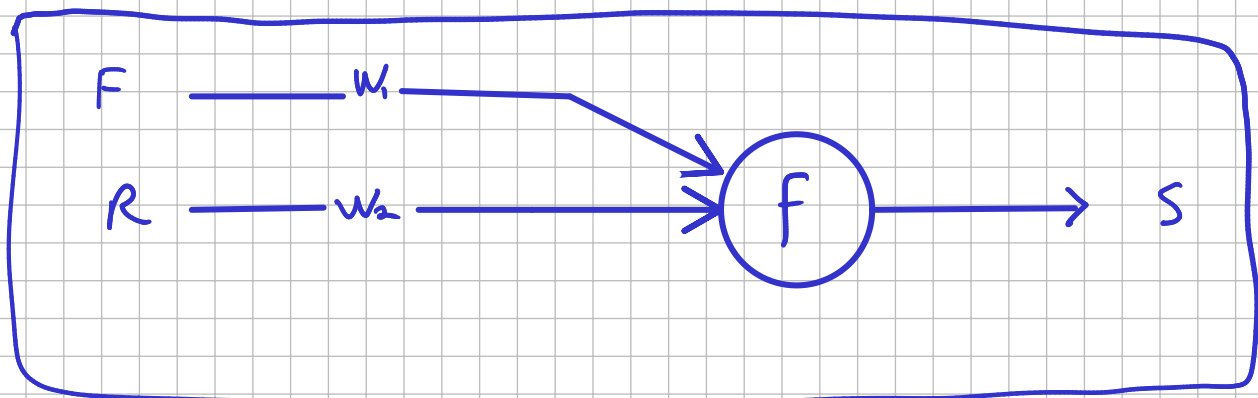
Im Beispiel das Autonomen Auto gibt es 2 binäre Sensoren:

Fussgänger vor dem Auto : F

Roter Ampel : R

In Logik: $(F \vee R) \rightarrow S$

$\Rightarrow$ Diese Funktion kann mit einem einzigen Perceptron implementiert werden.



$$w_1 = 1, w_2 = 1, b = -0.5$$

F	R	S
0	0	0
0	1	1
1	0	1
1	1	1

$$\begin{array}{l|l} \hat{y} = 0 & w^T x + b = 0 \cdot 1 + 0 \cdot 1 + (-0.5) = -0.5 < 0 \\ \hat{y} = 1 & w^T x + b = 0 \cdot 1 + 1 \cdot 1 + (-0.5) = 0.5 > 0 \\ \hat{y} = 1 & w^T x + b = 1 \cdot 1 + 0 \cdot 1 + (-0.5) = 0.5 > 0 \\ \hat{y} = 1 & w^T x + b = 1 \cdot 1 + 1 \cdot 1 + (-0.5) = 1.5 > 0 \end{array}$$

$\nearrow$
Linear Trennbar!

Regression

Für Regressionsprobleme sind nur dann nützlich, wenn die zu modellierenden Systeme linear sind.

⇒ Ein Lineares System S ist ein System, indem das Prinzip der Superposition gilt:

$$\alpha S(x) = S(\alpha x)$$

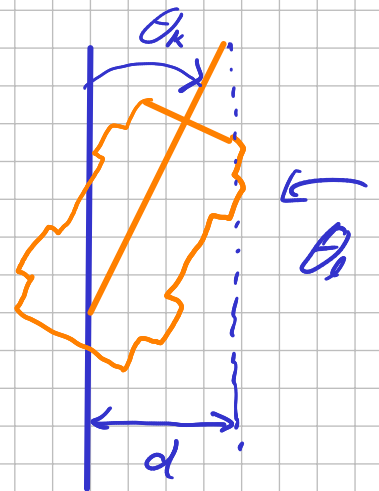
$$\text{und: } S(x_1 + x_2) = S(x_1) + S(x_2)$$

* α ist ein Skalar.

Beispiel: Autonomes Auto

In diesem Beispiel hat die selbstfahrenden Auto eine Kurskorrektur, wobei der aktuelle Kurs θ_k und der Abstand d der Vorderseite der Auto von der Mittellinie die Eingaben x_1 und x_2 sind. Die Ausgabe $\hat{y}$ ist der Lenkwinkel θ_l der zur Korrektur des Kurses erforderlich ist.

$x_1 = \theta_k$	$x_2 = d$	$\hat{y} = \theta_l$
-2	+0.4	-10
-1	+0.2	-5
+2	-0.3	+9
-4	+0.8	-20



Das System kann wie folgt beschrieben werden:

$$\hat{y} = S(x) \quad \text{wo: } x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Wir sehen ob das gilt:

$$\alpha S(x) = S(\alpha x)$$

⇒ Wenn die Eingänge und Ausgänge skalieren, weist das System Homogenität auf und ist wahrscheinlich linear!

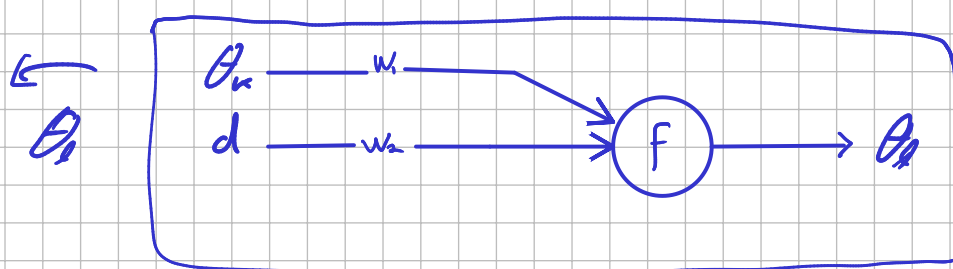
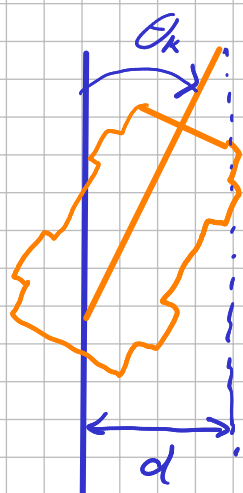
⇒ Dies kann getestet werden, indem man eine Zeile der Daten durch eine andere dividiert und einen Wert für α erhält

$x_1 = \theta_k$	$x_2 = d$	$\hat{y} = \theta_l$
-2	+0.4	-10
-1	+0.2	-5
+2	-0.3	+9
-4	+0.8	-20

$$\rightarrow \alpha x = \begin{pmatrix} \frac{-2}{-1} x_1 \\ \frac{0.4}{0.2} x_2 \end{pmatrix} = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix}$$

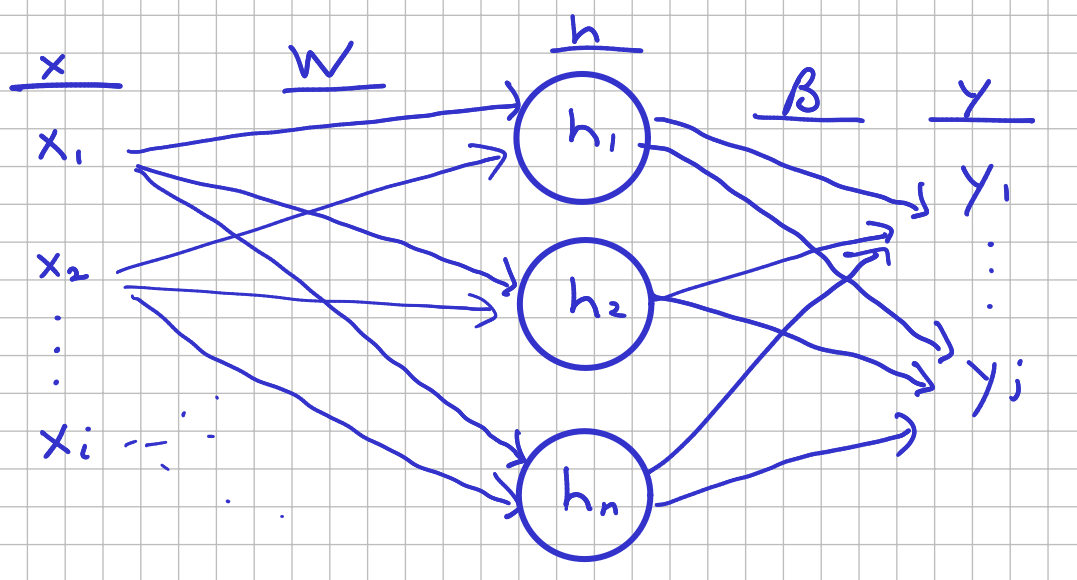
$$S(\alpha x) = \frac{-10}{-5} \hat{y} = 2 \hat{y}$$

$\Rightarrow$ Das Steuerungssystem für das Auto könnte also als Perceptron implementiert werden.



Multilayer Perceptron

Bei Daten für ein System das nichtlineare Merkmale aufweist - d.h. für einen Klassifikator mit nichtlinear trennbar Klassen oder ein System mit nichtlinear Regression - ist ein Perceptron nicht in die Lage, die durch das System beschriebene Funktion zu approximieren.



$$\begin{aligned} x &\in \mathbb{R}^{d_{in}} \\ W &\in \mathbb{R}^{d_{in} \times d_h} \\ h &\in \mathbb{R}^{d_h} \\ B &\in \mathbb{R}^{d_h \times d_{out}} \\ y &\in \mathbb{R}^{d_{out}} \end{aligned}$$

$$h = f(xW)$$

$$y = g(hB)$$

f, g sind differenzierbar!

Beispiel: Anomalieerkennung - Blinker

Ein Blinker im Computer eines Autos registriert, wenn der Blinkerhebel nach links L oder rechts R gedrückt wird. Wenn der Hebel nach links gedrückt wird, sollte der Blinker B blinken. Dasselbe für den rechten Hebel.

Wenn das System jedoch erkennt, dass der Hebel in beide Richtungen gleichzeitig gedrückt wird, liegt wahrsch. ein Fehler vor und der Blinker soll nicht aktiviert werden.

⇒ Das ist eine XOR Funktion.

Aussagelogik:

$$(L \vee R) \wedge \neg (L \wedge R) \rightarrow B$$

$$w_1 = 1, w_2 = 1$$

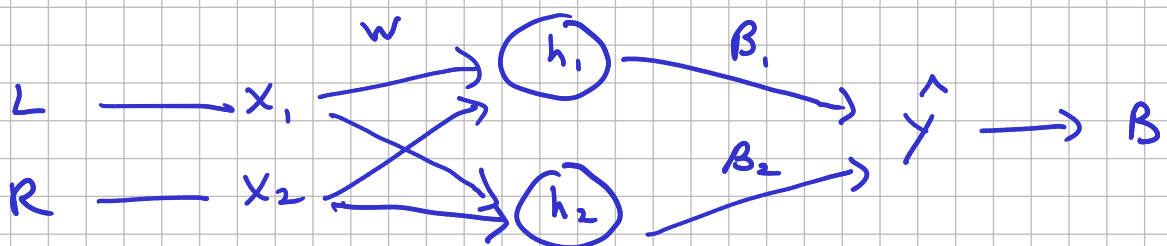
$$b = -0.5$$

Das binäre Klassifizierungsproblem ist nicht linear trennbar!

L	R	B	
0	0	0	$\hat{y} = 0 \quad w^T x + b = 0 \cdot 1 + 0 \cdot 1 + (-0.5) = -0.5 < 0$
0	1	1	$\hat{y} = 1 \quad w^T x + b = 0 \cdot 1 + 1 \cdot 1 + (-0.5) = 0.5 > 0$
1	0	1	$\hat{y} = 1 \quad w^T x + b = 1 \cdot 1 + 0 \cdot 1 + (-0.5) = 0.5 > 0$
1	1	0	$\hat{y} = 0 \quad w^T x + b = 1 \cdot 1 + 1 \cdot 1 + (-0.5) = 1.5 > 0$

Beispiel - Anomalieerkennung

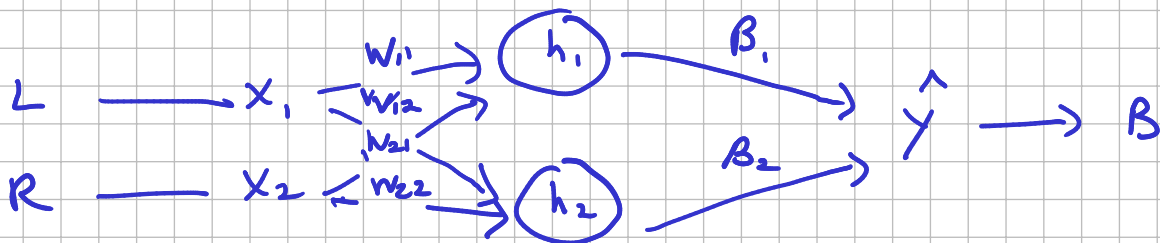
Die folgende MLP könnte die XOR Funktion implementieren:



Training und Validation

Algorithmus:

1. Loss function definieren $\mathcal{L}(\hat{y}, y) = y - \hat{y}$
2. Learning Rate definieren $\eta = 0.1$
3. While (Loss nicht akzeptabel ist):
 - Berechnen der Ausgabe $\hat{y}$ für die Eingaben x_1 und x_2
 - Berechnen der Loss
 - Gewichte updaten: $\forall i: w_i = w_i + \eta L$



- ⇒ In diesem Fall müssen wir Backpropagation verwenden.
- Dazu wird ein Forward-Pass durchgeführt
 - der Gradient der Loss wird berechnet
 - rückwärts durch das Netz propagiert um die Gewichte zu aktualisieren.

1. Forward-Pass

$$h_1 = f(w_{11}x_1 + w_{12}x_2)$$
$$h_2 = f(w_{21}x_1 + w_{22}x_2)$$
$$\hat{y} = g(\beta_1 h_1 + \beta_2 h_2)$$

2. Loss berechnen: $\mathcal{L} = \frac{1}{2} (\hat{y} - y)^2$
(MSE)

* Frage: Wie können wir die Gewichte so ändern, dass der Verlust reduziert wird?

3. Berechnen Sie den Gradienten in Bezug auf die Output layer Gewichte.

Bsp: β_1

$$\frac{\partial L}{\partial \beta_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial (\beta_1 h_1 + \beta_2 h_2)} \cdot \frac{\partial (\beta_1 h_1 + \beta_2 h_2)}{\partial \beta_1}$$

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

$$\frac{\partial \hat{y}}{\partial (\beta_1 h_1 + \beta_2 h_2)} = g'(\beta_1 h_1 + \beta_2 h_2)$$

$$\frac{\partial (\beta_1 h_1 + \beta_2 h_2)}{\partial \beta_1} = h_1$$

Bsp. w_{11}

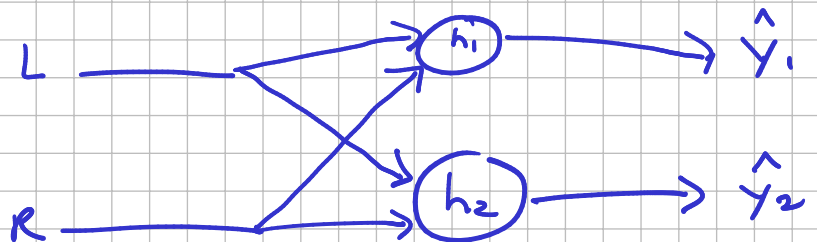
$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial (\beta_1 h_1 + \beta_2 h_2)} \cdot \frac{\partial (\beta_1 h_1 + \beta_2 h_2)}{\partial h_1} \cdot \frac{\partial h_1}{\partial (w_{11} x_1 + w_{12} x_2)} \cdot \frac{\partial (w_{11} x_1 + w_{12} x_2)}{\partial w_{11}}$$

Kettenregel

$$\frac{dh}{dx} = \frac{dh}{dg} \cdot \frac{dg}{d}$$

Aufgabe (Python)

L	R	B
0	0	0
0	1	1
1	0	1
1	1	0



One-hot:

$$\begin{pmatrix} y_1 \\ y_2 \end{pmatrix}$$

Bsp: $0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$
 $1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

H4 verwendet nur dot-product - kein Gradienten.
(Skalar-produkt)

$\Rightarrow$ versucht ihr diese Problem zu implementieren